Project 2 – Acme Analytics Linux Platform

Day 2 · Phase 1, Steps 1.5–1.9: ports, SSH keys, links and inodes, users and groups, Linux vs cloud NAT

Muhammad Bilal Javed · 15–16 October 2026

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	1 – Network foundation: 1.5 ports and sockets · 1.6 SSH keys · 1.7 links and inodes · 1.8 users and groups · 1.9 NAT comparison
Dates	15–16 October 2026
Hosts worked on	app01, stor01, gw01
Status	Step 1.5 ✅ · Step 1.6 ✅ – password-free SSH, remote commands and scp from gw01 to both servers · Step 1.7 ✅ – hard links, soft links, inodes · Step 1.8 ✅ – users, groups, service account · Step 1.9 ✅ – NAT comparison · **Phase 1 complete, snapshot taken**
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the commands, with a comment on every line. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary.

---

1. Objective

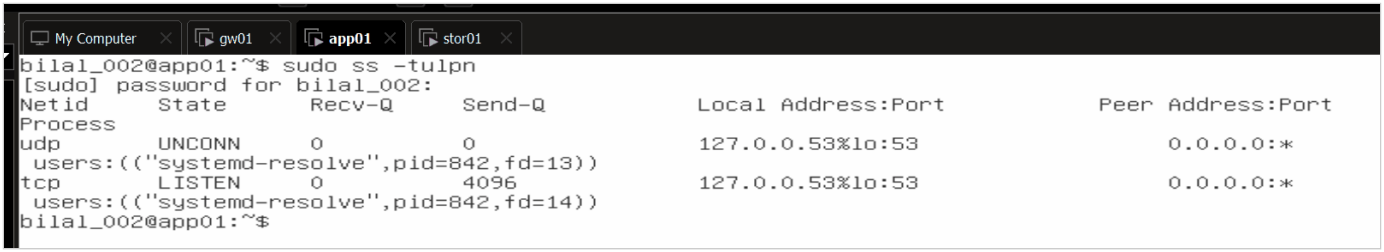
💡 **In plain words:** an IP address is the **address of a building** (which machine). A **port** is a **door number** inside that building (which program). Every service waits behind its own numbered door: SSH behind door 22, DNS behind door 53, a website behind door 80. Before opening new doors, I made an inventory of which doors app01 already has open – and to whom.

🔧 **Technical:** list all listening TCP and UDP sockets on app01 with the owning process, and interpret the result.

2. Procedure

```
sudo ss -tulpn   # ss = socket statistics
#  -t  TCP sockets        -u  UDP sockets
#  -l  only listening      -p  show the owning process
#  -n  numbers, not names  (memory aid: "tulpn" = tulip)
```

📸 **Evidence:**



ss -tulpn on app01

3. Reading the output

Column	Technical meaning	In plain words	In my output
Netid	Protocol: TCP or UDP	TCP = a phone call (confirmed); UDP = a postcard (just sent)	`udp` and `tcp`
State	LISTEN = TCP socket waiting for connections; UNCONN = open UDP socket (UDP never connects); ESTAB = an active TCP connection	LISTEN = a door open and waiting; UNCONN = a letterbox ready for letters	`UNCONN` (udp), `LISTEN` (tcp)
Recv-Q	Bytes received but not yet read by the program	Letters that arrived but are not opened yet	`0` – nothing waiting
Send-Q	On a LISTEN socket: the maximum queue of waiting connections	How many visitors may queue at the door	`4096` on the DNS helper's TCP socket; `0` for UDP (no queue)
Local Address:Port	IP and port on this machine where the socket listens	Building address + door number	`127.0.0.53%lo:53`
Peer Address:Port	The other end of a connection	Who is visiting	`0.0.0.0:*` – nobody yet, anyone may come
Process	Program name, process ID, file descriptor	Which program sits behind the door	`systemd-resolve`, pid 842, fd 13 (UDP) and fd 14 (TCP)

The address in detail: `127.0.0.53 % lo : 53` = the loopback address (127.x = the machine talking to itself), bound to the loopback card `lo`, on port 53 (DNS). Reachable **only from programs inside app01**.

fd (file descriptor): the number a program uses for something it has open – in Linux almost everything is a file, including network sockets. fd 13 and fd 14 are the DNS helper's two sockets.

4. TCP and UDP – the background

	TCP	UDP
Start	**3-way handshake:** SYN → SYN-ACK → ACK ("Can you hear me?" – "Yes, can you hear me?" – "Yes")	No handshake – it just sends
Lost data	Noticed and re-sent automatically – reliable	Lost for good – no check
Speed	Slower, but sure	Faster, no guarantee
In ss	LISTEN → ESTAB	UNCONN
Used for	SSH (22), websites (80/443)	DNS (53), video calls, games

DNS uses UDP for short questions and TCP for answers too large for one packet – that is why the DNS helper listens on both. The firewall rule `ESTABLISHED,RELATED` from Step 1.3 means exactly "packets belonging to a TCP conversation that already completed its handshake".

5. Final verdict

What	Result
Open ports on app01	**1:** port 53 (DNS), UDP + TCP
Program	`systemd-resolved` – the local DNS helper
Reachable from the network?	❌ No – bound to loopback only
SSH (port 22)	❌ Not listening – the package was installed but the service was not enabled
Security view	✅ Minimal attack surface – app01 exposes nothing to the network
Impact on the project	⚠️ No remote administration possible: SSH keys (Step 1.6) and Ansible (Phase 7A) need SSH

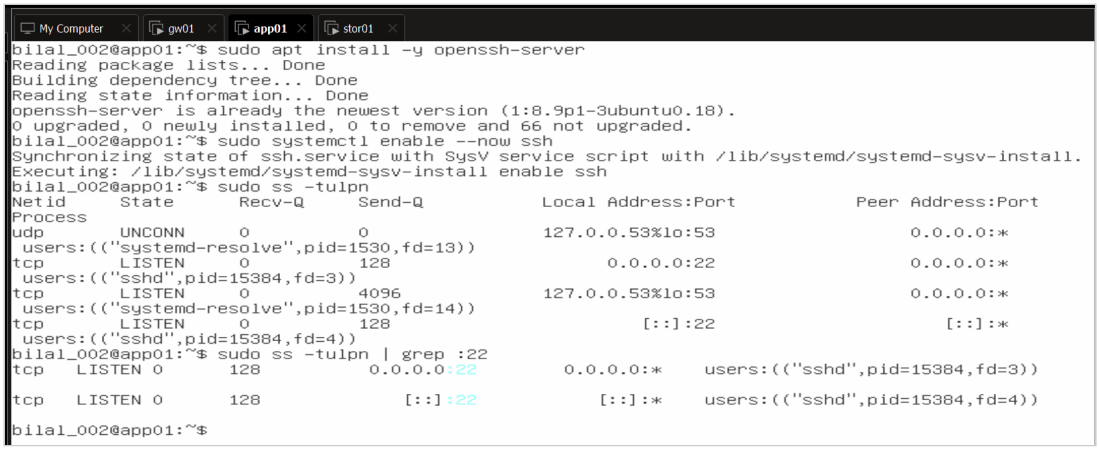
> **Before:** app01 exposed no network services at all – only the local DNS resolver on 127.0.0.53:53, reachable from inside the machine. The SSH package was installed but its service was not running, so it had to be enabled for remote administration.

6. Fix: enable SSH on app01

🔧 **Commands:**

```
sudo apt install -y openssh-server   # make sure the SSH server package is installed
sudo systemctl enable --now ssh      # start it now (--now) AND at every boot (enable)
sudo ss -tulpn | grep :22            # verify: is anything listening on door 22?
```

📸 **Evidence:**



SSH enabled on app01, listening on port 22

Line	Meaning
`openssh-server is already the newest version`	The package was already installed – only the **service was not running**. `enable --now` started it and made it start at every boot
`tcp LISTEN 0.0.0.0:22 … sshd`	SSH waits on door 22 on **all IPv4 addresses** – reachable from gw01 and stor01
`tcp LISTEN [::]:22 … sshd`	The same for IPv6 (`[::]` = all IPv6 addresses)
Send-Q 128	sshd's waiting queue – up to 128 connections may queue
`systemd-resolve` pid 1530 (before: 842)	New process number after the VM restart – normal

Before → after: app01 went from **0 doors open to the network** to **1** – SSH on port 22.

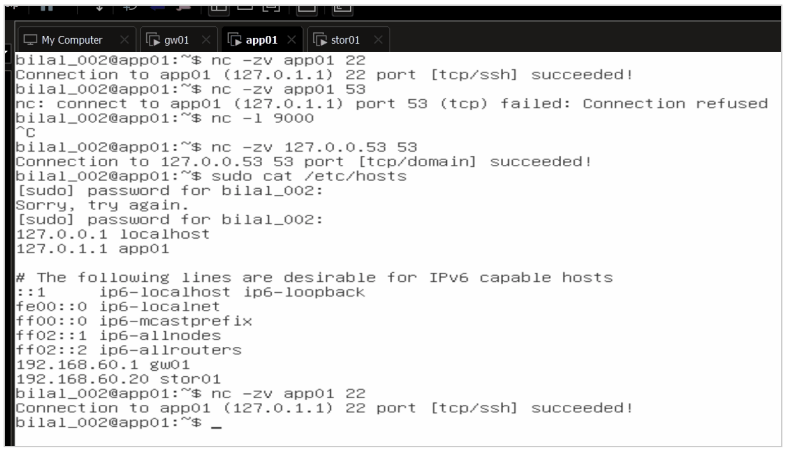
7. Testing a port: `nc -zv` and the binding address

💡 **In plain words:** ping only proves a machine is alive. To prove a **service** answers, you knock on its **door** (port). `nc` (netcat) does exactly that – the admin's standard check when "ping works but the application doesn't".

🔧 **Commands (on app01 – the machine calling itself):**

```
nc -zv app01 22        # -z: only check the door, send no data · -v: print the result
nc -zv app01 53        # same for door 53
nc -zv 127.0.0.53 53   # door 53 at the DNS helper's own address
cat /etc/hosts         # where the name "app01" points to
```

📸 **Evidence:**



Port tests on app01 with nc -zv

Test	Result	Why
`app01:22` (= 127.0.1.1)	✅ succeeded	sshd listens on `0.0.0.0` = **every** address of the machine
`app01:53` (= 127.0.1.1)	❌ Connection refused	The DNS helper listens **only on 127.0.0.53** – a different address
`127.0.0.53:53`	✅ succeeded	The right address – the DNS service **is** running

Three loopback addresses (the whole range 127.x.x.x means "this machine itself", all on the card `lo`):

Address	Created by	Seen in	Purpose
`127.0.0.1`	Worldwide standard	`/etc/hosts` → `localhost`	"Myself" for any program
`127.0.1.1`	Ubuntu installer (hostname)	`/etc/hosts` → `app01`	The machine finds its own name without a network
`127.0.0.53`	systemd-resolved	`/etc/resolv.conf` → `nameserver`	The DNS helper's private address

Refused vs. timeout – two different failures:

Result	What happened	Usual cause
Connection refused	The machine answered "nobody here"	Service not running, or listening on another address
Timeout	No answer at all	Firewall silently dropping, network broken, machine off

Refused points to the **service**; *timeout* points to the **network or firewall**.

8. Key understanding (in my own words)

> **Netid** is the protocol, TCP or UDP. **TCP** starts every conversation with a 3-way handshake (SYN, SYN-ACK, ACK); after that the connection is **ESTAB**. **UDP** has no handshake – it just sends.

Every port leads directly to its own program. The port number decides which program receives the traffic: port **22** → **sshd** (SSH), port **53** → the **DNS helper**, port **80** → a web server. Traffic does **not** pass through the DNS helper first. A port with no program behind it is closed – nobody answers.

LISTEN = a TCP door is open and waiting. **UNCONN** = a UDP door is open. **Recv-Q** = data received but not read yet (0 = nothing waiting). **Send-Q** on a LISTEN socket = how many visitors may wait in the queue (4096 for the DNS helper's TCP socket). **Local address** = this machine's address and door, e.g. `127.0.0.53%lo:53` = the DNS helper on the loopback card, port 53, reachable only from inside app01. **Peer address** = who is visiting – `0.0.0.0:*` = nobody yet; in a real connection it shows the visitor's IP and its random port, e.g. `192.168.60.1:51234` = gw01.

A server can test itself by calling its own address – loopback (127.x.x.x). `localhost` (127.0.0.1) and `app01` (127.0.1.1) both mean "myself". SSH listens on **all** addresses (`0.0.0.0`), so `localhost:22` and `app01:22` both work. The DNS helper listens on **one fixed address only, 127.0.0.53**, so only `127.0.0.53:53` works – any other address gets "connection refused". **The binding address matters:** `0.0.0.0` = the door is open at every entrance of the building; `127.0.0.53` = the door exists at one entrance only.

A port is a door number; a socket is a program standing at that door on a specific machine (IP + port + protocol); a connection is two sockets talking to each other.

DNS has one job: translate a name into an IP address – the phone book, not the phone line. Lookup order on app01: **`/etc/hosts`** (own contact list) → **local DNS helper** `127.0.0.53` (answers from its cache) → **outside DNS server** `1.1.1.1` / `8.8.8.8` through gw01. Once the IP is known, the real traffic goes directly to that IP and port. In a company, an **internal DNS server** answers internal names; in our lab `/etc/hosts` does that job.

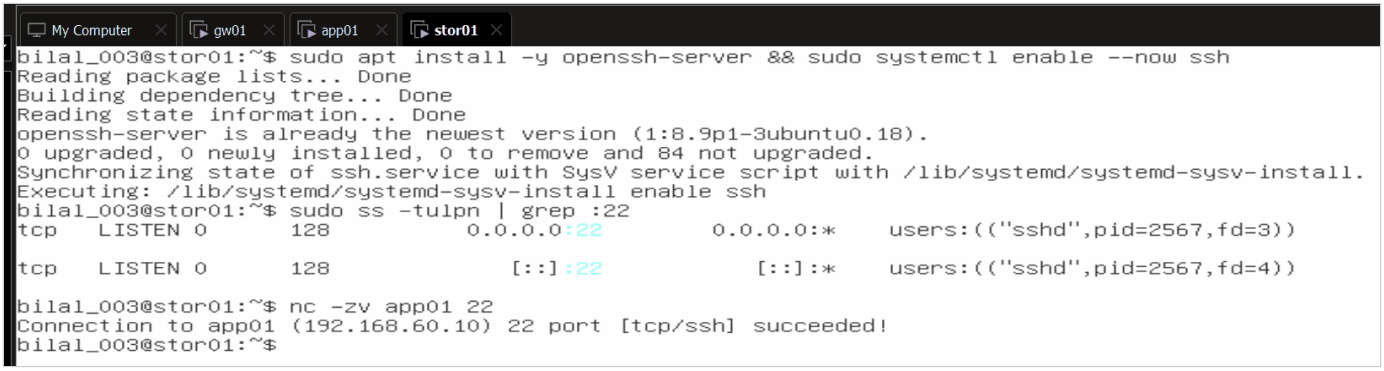
Before the fix, the only open port on app01 was 53, used by the local DNS helper, and only from inside app01 – it accepted **no connections from the network**. Next: enable SSH so port 22 opens.

9. Step 1.5 finished – SSH reachable across the lab network

🔧 **On stor01:** enable SSH and test app01 **from another machine** (on gw01 SSH was already running; only checked):

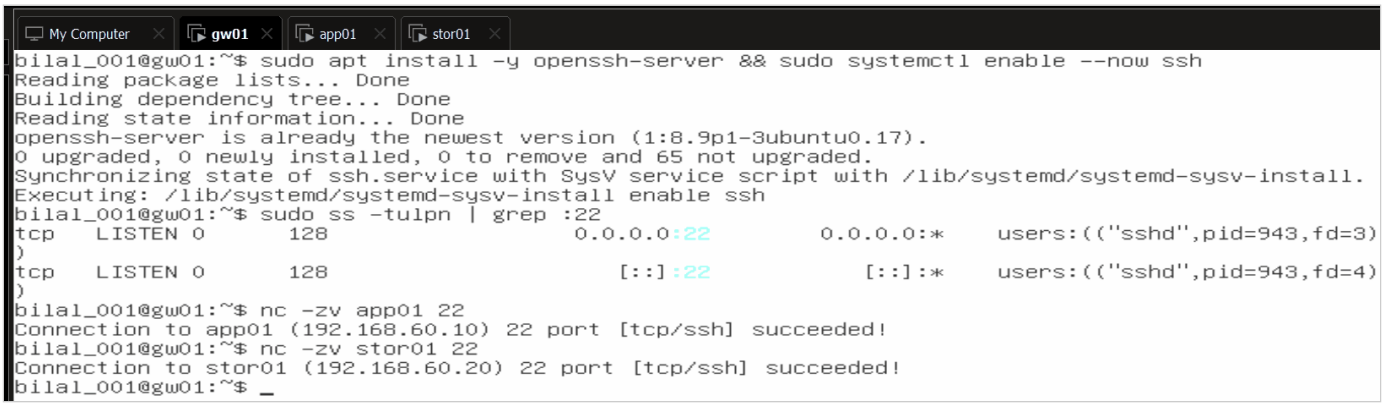
```
sudo apt install -y openssh-server && sudo systemctl enable --now ssh
sudo ss -tulpn | grep :22     # LISTEN 0.0.0.0:22 = door 22 open to the network
nc -zv app01 22               # knock on app01's SSH door over the lab network
```

📸 **Evidence – stor01:** SSH listening, and app01 reached at **192.168.60.10** (not loopback):



stor01: SSH enabled, nc to app01 succeeded

📸 **Evidence – gw01:** SSH listening, both servers reachable on port 22:



gw01: SSH listening, nc to app01 and stor01 succeeded

From	To	Result
stor01	app01 (192.168.60.10):22	✅ succeeded
gw01	app01 (192.168.60.10):22	✅ succeeded
gw01	stor01 (192.168.60.20):22	✅ succeeded

Step 1.5 complete: every server runs SSH on port 22 (`0.0.0.0` and `[::]`), reachable across the lab network.

10. Step 1.6 – SSH keys: log in without a password

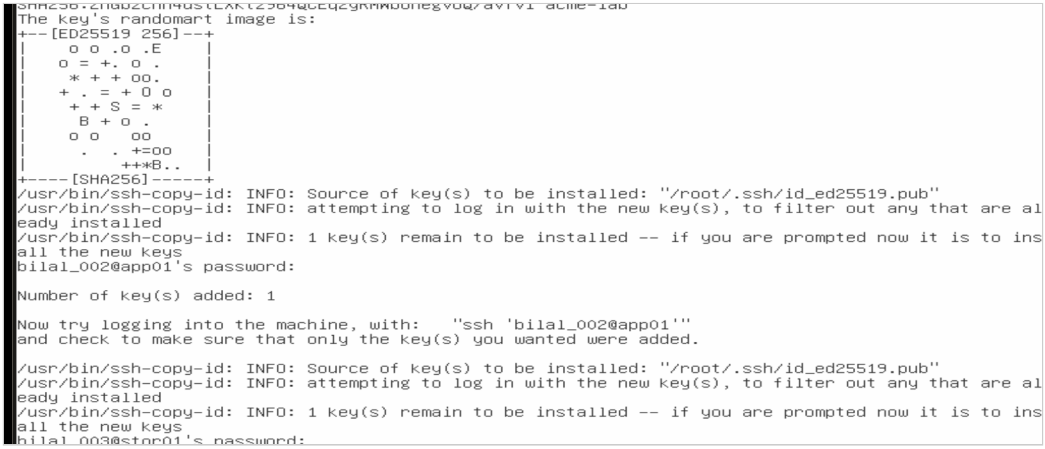
💡 **In plain words:** a password is something you **type**; an SSH key is a **pair of files** – a private key that never leaves gw01 (like a house key in your pocket) and a public key that is copied to each server (like a lock fitted to the door). If the key fits the lock, the door opens – no password. gw01 needs this because it becomes the **Ansible control node** in Phase 7A and must reach every server automatically.

🔧 **On gw01, as the normal user bilal_001 (not root):**

```
ssh-keygen -t ed25519 -C "acme-lab"   # create the key pair (Enter 3×: default path, no passphrase for the lab)
ssh-copy-id bilal_002@app01           # copy the PUBLIC key to app01 (asks the password once)
ssh-copy-id bilal_003@stor01          # same for stor01
```

Issue: key first created for root

📸 The first attempt ran as root – the key landed in `/root/.ssh/`, so password-free login would only work with `sudo ssh …`:



ssh-copy-id using /root/.ssh/id_ed25519.pub

Fix: repeated as **bilal_001**, without `sudo`. The key now lives in `/home/bilal_001/.ssh/`. (The root key copied earlier will be removed during hardening in Phase 6 – servers should not trust gw01's root user.)

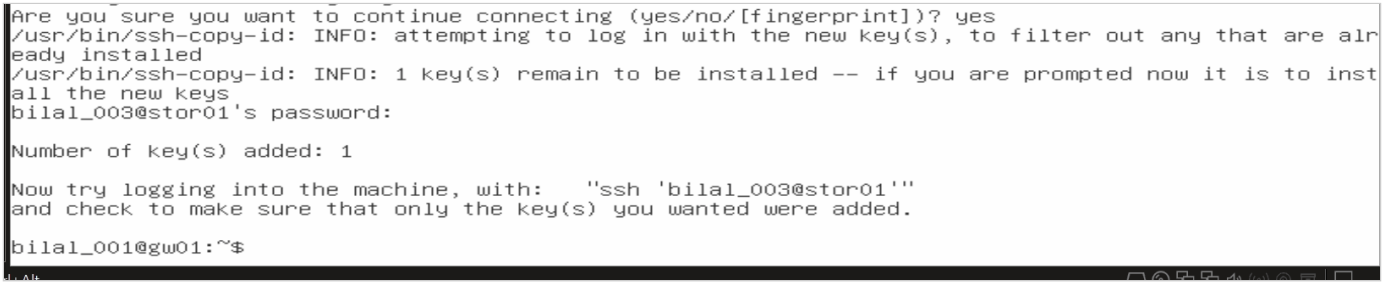
📸 **Evidence – correct key for bilal_001:**



ssh-keygen as bilal_001, key in /home/bilal_001/.ssh

"The authenticity of host … can't be established" – first visit to a server: gw01 asks whether to trust the server's ID (fingerprint). After `yes` it is stored in `~/.ssh/known_hosts`; if a server's ID ever changes, SSH warns – protection against impostors.

📸 **Evidence – key added to stor01 ("Number of key(s) added: 1"):**

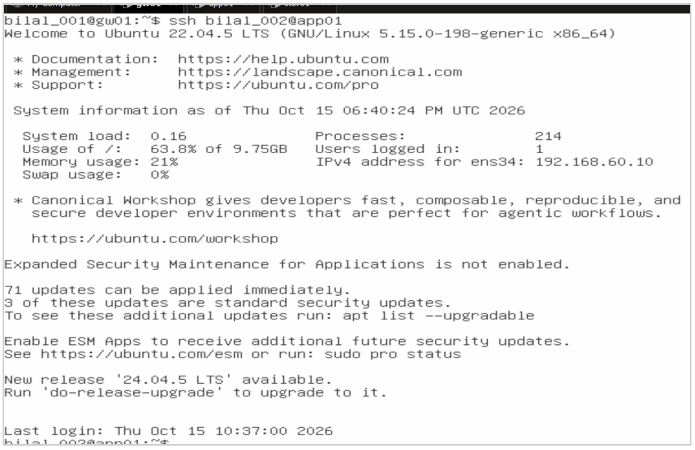


ssh-copy-id to stor01

The proof: login without a password

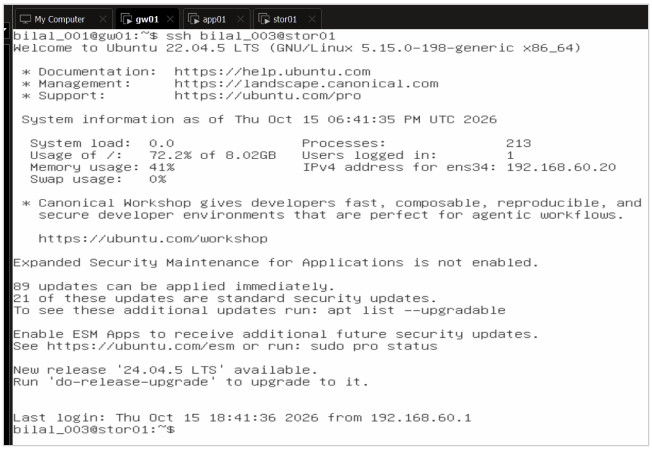
```
ssh bilal_002@app01     # log in to app01 — no password prompt
exit                    # close the session, back to gw01
ssh bilal_003@stor01    # log in to stor01 — no password prompt
exit                    # back to gw01
```

📸 **app01** – logged in with no password; the welcome screen confirms `IPv4 address for ens34: 192.168.60.10`:



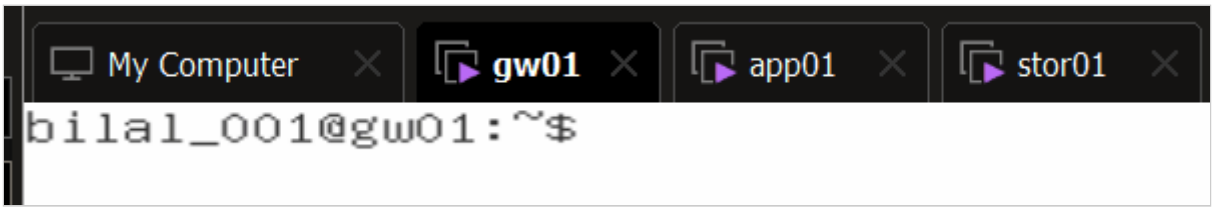
SSH login to app01 without password

📸 **stor01** – same; note `Last login … from 192.168.60.1` – Linux records who logged in from where (the start of an audit trail):



SSH login to stor01 without password

📸 **`exit`** closes the remote session – the prompt `bilal_001@gw01:~$` shows I am back on gw01:



Back on gw01 after exit

Habit: always read the prompt before typing – `user@host` tells you which server the next command runs on.

Observed on the welcome screens	Meaning
app01 / 63.8 % of 9.75 GB · stor01 / 72.2 % of 8.02 GB	stor01's system disk is filling – data moves to the separate LVM disks in Phase 2
71 (app01) / 89 (stor01) updates, 3 / 21 security	to be applied during hardening (Phase 6)
"New release 24.04.5 available"	deliberately **not** upgraded – the lab stays on 22.04

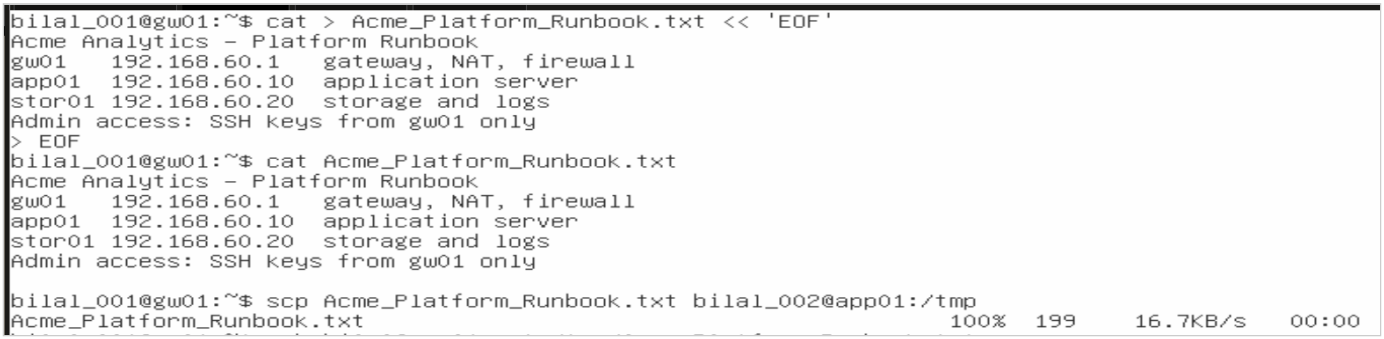
Copying a file over SSH: `scp`

💡 **In plain words:** `scp` (secure copy) sends a file to another server through the same encrypted SSH door – and because the key is installed, no password is needed. Instead of a throw-away test file I created a short **platform runbook** – the kind of handover note admins keep.

```
cat > Acme_Platform_Runbook.txt << 'EOF'      # write the lines below into the file …
Acme Analytics - Platform Runbook
gw01   192.168.60.1   gateway, NAT, firewall
app01  192.168.60.10  application server
stor01 192.168.60.20  storage and logs
Admin access: SSH keys from gw01 only
EOF                                           # … until the end marker EOF ("End Of File")
scp Acme_Platform_Runbook.txt bilal_002@app01:/tmp/        # copy it to app01's /tmp folder
ssh bilal_002@app01 cat /tmp/Acme_Platform_Runbook.txt     # run ONE command on app01 and show the result
```

Heredoc (`<< 'EOF'`): "write everything I type until you see EOF". The closing `EOF` stands alone on its line; the quotes keep the text exactly as typed. File names use underscores – spaces in Linux file names need quoting and break scripts.

📸 **Evidence – file created and copied (`100% 199 bytes`, no password):**



Runbook created with a heredoc and copied with scp

📸 **Evidence – read back on app01 with a one-shot SSH command:**



Runbook read on app01 via ssh

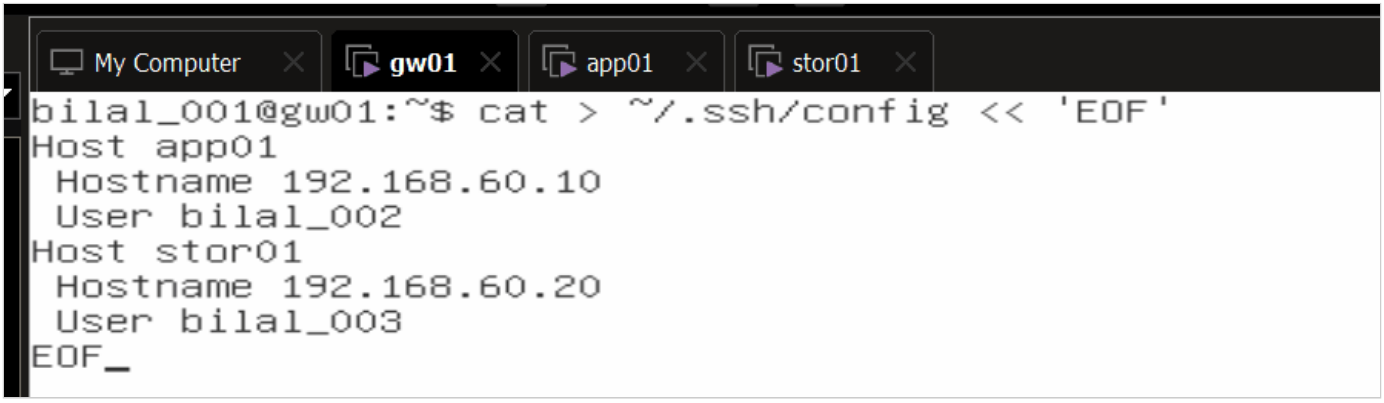
`ssh user@host command` runs a single command on the remote server and returns – the same pattern Ansible uses later.

SSH shortcuts: `~/.ssh/config`

💡 **In plain words:** a personal address book for SSH – type `ssh app01` instead of `ssh bilal_002@192.168.60.10`.

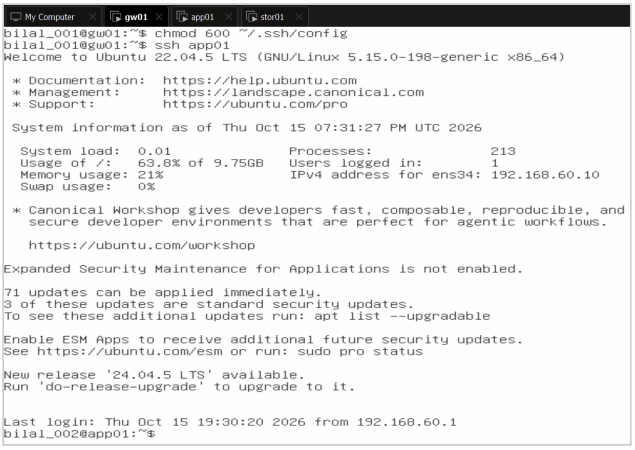
```
cat > ~/.ssh/config << 'EOF'
Host app01
    HostName 192.168.60.10
    User bilal_002
Host stor01
    HostName 192.168.60.20
    User bilal_003
EOF
chmod 600 ~/.ssh/config   # only the owner may read it – SSH refuses a config others can read
ssh app01                 # log in by short name – no user, no password
exit
ssh stor01
exit
```

📸 **Evidence – the shortcut file:**



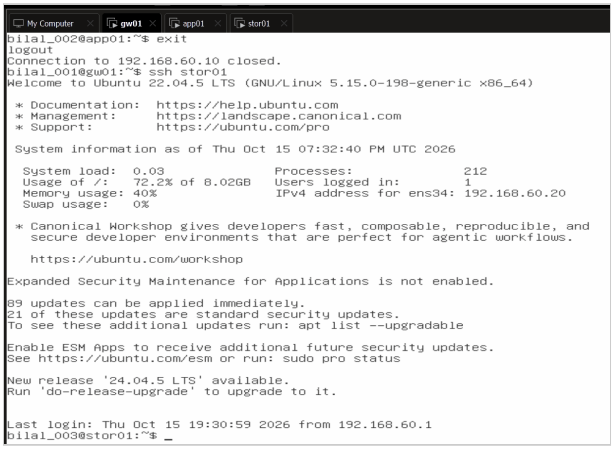
~/.ssh/config with Host entries for app01 and stor01

📸 **Evidence – `ssh app01` logs in as bilal_002 without a password:**



ssh app01 by alias

📸 Evidence – `exit` ("Connection to 192.168.60.10 closed"), then `ssh stor01`:



exit and ssh stor01 by alias

Interactive transfer: `sftp`

```
sftp app01                          # open a file session over SSH
sftp> ls                            # list files in the home folder on app01
sftp> cd /tmp                       # change folder on app01
sftp> get Acme_Platform_Runbook.txt # download to gw01
sftp> exit
```

📸 Evidence – runbook downloaded from app01 to gw01 (100 %):



sftp session: ls, cd, get

Why not FTP?

> **FTP sends usernames, passwords and files in clear text** – anyone listening on the network can read them. **SCP and SFTP run inside SSH**, so everything is **encrypted**, and with keys no password crosses the network at all. Companies do not use plain FTP for administration.

Tool	Use	Encrypted
`scp`	copy one file quickly	✅ (SSH)
`sftp`	browse and transfer interactively	✅ (SSH)
`ftp`	legacy file transfer	❌ clear text

Step 1.6 complete: gw01 reaches app01 and stor01 by key and by short name, runs remote commands and copies files with `scp` and `sftp` – all encrypted, all without a password.

11. Step 1.7 – Hard links, soft links and inodes

💡 **In plain words:** every file has an **ID card** (the *inode*) holding its owner, permissions, size, dates and where the data sits on disk. The **file name is only a label** in a folder pointing to that ID card. - A **hard link** = a **second label on the same ID card**. Delete one label, the file lives on. - A **soft link** = a **signpost** with its own ID card that only says “go to that name”. Delete the original, the signpost points to nothing.

🔧 **Technical:** a directory is a list of *name → inode number*. `ln` adds another directory entry for the same inode (link count +1). `ln -s` creates a new inode of type symlink that stores a path. The inode and its data blocks are freed only when the link count reaches 0 **and** no process still has the file open.

Why in this project: Phase 2 backs up app data to stor01 with `rsync --link-dest`. Every day’s backup folder hard-links the files that did not change to yesterday’s copy, so 30 daily backups look full-size but take little extra space. A soft link `/backups/latest` points to the newest day and is moved each night. The same signpost idea is used by nginx (`sites-enabled → sites-available`) in Phase 7. This lab proves the mechanism on real files first.

Procedure (on app01, as bilal_002)

```
mkdir -p ~/acme && cd ~/acme
echo "APP_PORT=8080" > app.conf          # an Acme config file
ln app.conf app.conf.backup              # hard link: second name, same inode
ln -s app.conf current.conf              # soft link: signpost holding the path "app.conf"
ls -li                                   # first column = inode number, third = link count
echo "LOG_LEVEL=info" >> app.conf.backup # write through the hard link
cat app.conf                             # the change appears here too — same file
stat app.conf | head -3                  # Inode and Links: 2
```

📸 **Evidence – same inode 276722 and link count 2 for both names; the soft link has its own inode 276727; an edit via the hard link shows in the original:**

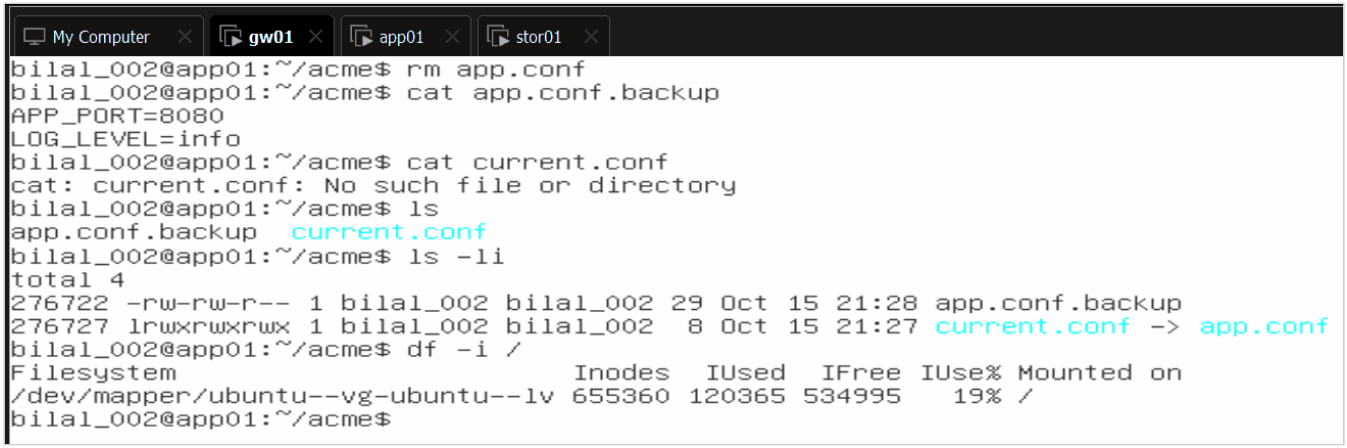
```
bilal_002@app01:~/acme$ ls -li
total 8
276722 -rw-rw-r-- 2 bilal_002 bilal_002 14 Oct 15 21:26 app.conf
276722 -rw-rw-r-- 2 bilal_002 bilal_002 14 Oct 15 21:26 app.conf.backup
276727 lrwxrwxrwx 1 bilal_002 bilal_002  8 Oct 15 21:27 current.conf -> app.conf
bilal_002@app01:~/acme$ echo "LOG_LEVEL=info" >> app.conf.backup
bilal_002@app01:~/acme$ cat app.conf
APP_PORT=8080
LOG_LEVEL=info
bilal_002@app01:~/acme$ stat app.conf | head -3
  File: app.conf
  Size: 29              Blocks: 8          IO Block: 4096   regular file
Device: fd00h/64768d    Inode: 276722      Links: 2
bilal_002@app01:~/acme$
```

ls -li, write through the hard link, stat

The “accidental delete” test

```
rm app.conf           # simulate a colleague deleting the file
cat app.conf.backup   # data is still there
cat current.conf      # "No such file or directory" — the signpost is broken
ls -li                # link count of the survivor is now 1; current.conf shown in red
df -i /               # inode capacity of the root file system
```

📸 **Evidence – hard link survives, soft link broken, inode usage 19 %:**



rm original, hard link survives, soft link broken, df -i

Reading the results

Observation	Meaning
`276722 … 2 … app.conf` and `276722 … 2 … app.conf.backup`	One file, two names (link count 2)
`276727 lrwxrwxrwx … current.conf -> app.conf`	Separate inode; type `l` = symlink; size 8 = the length of the text "app.conf"
`LOG_LEVEL=info` appears in `app.conf`	A hard link is **not a copy** – an edit through either name changes the one file
After `rm`: `app.conf.backup` still readable, link count **1**	Data is freed only when the last name is gone
`current.conf` shown in red, `cat` fails	Dangling soft link – it points to a name that no longer exists
`df -i /`: 655,360 inodes, 120,365 used, **19 %**	Each file uses one inode; when inodes run out you get "No space left on device" even with free GB – worth monitoring (Phase 5A)

Key understanding (in my own words)

> Linux works with inode numbers; the name I type is only a label in a folder. A **hard link** is a second label on the same inode – deleting one name loses nothing, but it does **not** protect against bad edits, because it is the same file. A **soft link** has its own inode and only holds a path – deleting it never harms the original, but deleting the original breaks it. Hard links protect important files against accidental deletion; real protection against bad changes still needs backups (Phase 2).

Step 1.7 complete. `~/acme` stays on app01 as sample data.

12. Step 1.8 – Users, groups and a service account (continued 2026-10-16)

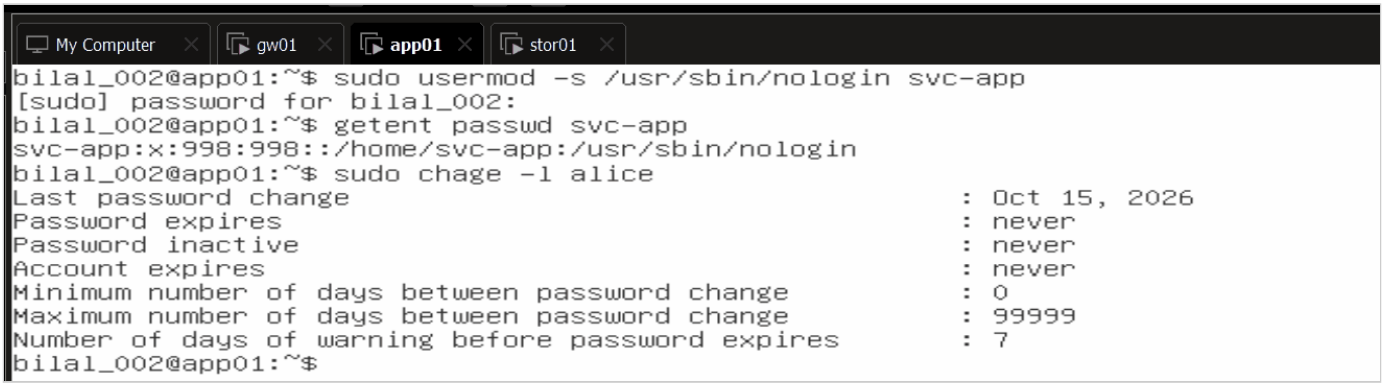
💡 **In plain words:** every person gets their own account, people who do the same job share a **group**, and every **program** gets its own low-rights account that nobody can log in to. When someone leaves, their account is **locked**, not shared or forgotten.

Why in this project: the security questionnaire in the brief asks "who can log in, and with which rights?". This step answers it: named users, a team group, and a no-login service account. In Phase 3 the Acme app's systemd service runs as `svc-app`; in Phase 6 groups decide who may use `sudo` and Docker; in Phase 7A Ansible automates exactly this joiner/leaver process.

Procedure (on app01)

```
sudo groupadd devs                                  # team group (it already existed)
sudo useradd -m -s /bin/bash -G devs alice          # -m home folder, -s shell, -G extra group
sudo useradd -m -s /bin/bash -G devs bob
sudo useradd -r -s /usr/sbin/nologin svc-app        # -r system account, nobody can log in
sudo passwd alice                                   # give alice a password
id alice                                            # uid, primary group, secondary groups
getent group devs                                   # members of devs
sudo chage -l alice                                 # password aging ("change age")
sudo usermod -L bob                                 # lock (leaver)
sudo passwd -S bob                                  # status: L = locked, P = password, NP = none
sudo usermod -U bob                                 # unlock (returner)
```

📸 **Evidence – svc-app has the shell `/usr/sbin/nologin` and system uid 998; alice's password aging:**



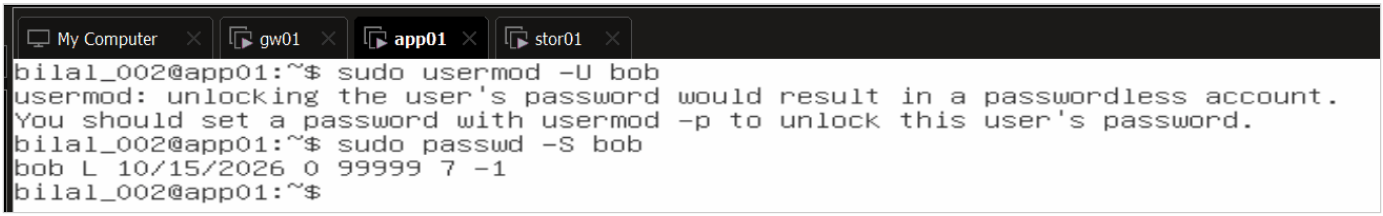
getent passwd svc-app and chage -l alice

📸 **Evidence – devs = alice, bob; svc-app in its own group only; bob locked (L):**



getent group devs, id svc-app, lock bob

📸 **Evidence – unlock refused, because bob has no password yet:**



usermod -U refused: passwordless account

Reading the results

Output	Meaning
`uid=1001(alice) gid=1002(alice) groups=1002(alice), 1001(devs)`	Primary group = her own private group; secondary group = devs
`devs:x:1001:alice,bob`	Both developers are in the team group
`svc-app:x:998:998::/home/svc-app:/usr/sbin/nologin`	uid **below 1000** = system/service account; shell `nologin` = no human login
`chage -l alice`: expires never, max 99999	No password expiry yet – to be set by policy in Phase 6 (e.g. `chage -M 90`)
`bob L 10/15/2026 0 99999 7 -1`	Status **L** = locked; then min/max/warn days and inactive (-1 = not set)
`usermod -U bob` refused	Safety feature: unlocking an account without a password would leave it open with **no password at all**. Set a password first (`sudo passwd bob`), then lock/unlock.

Option cheat sheet

Option	Meaning
`useradd -m`	make the home folder
`useradd -s` / `usermod -s` (small s)	set the shell
`useradd -G`	add to extra groups
`useradd -r`	system account (uid < 1000, no home folder) – low rights come from no extra groups and not being root
`usermod -L` / `-U`	lock / unlock
`passwd -S` (capital S)	show the password status
`chage -l` / `-M 90`	show aging / force a new password every 90 days

Key understanding – final summary

> On app01, the server where the Acme application runs, we created named accounts for each developer – alice and bob – each with their own home folder (`-m`) and login shell (`-s`), and put both in a shared team group `devs`, so rights are given to the team instead of to each person; each user has a primary group (their own) and a secondary group (devs). We then created `svc-app` as a **system account** (`-r`, uid 998, below 1000) with the shell `nologin` and no password: no human can log in as it, it exists only so a program can run under it. Every service runs as some user – the main sshd as root with its pre-login part as `sshd`, nginx as `www-data` – and giving each service its own low-rights account means a hacked service reaches only that account's own files, not root. `chage` shows and sets the password-aging rules, and `usermod -L` / `-U` lock and unlock an account – the leaver/returner process. Linux refused to unlock bob while he had no password, because a human account must have a password or key before it is opened. This step answers the Acme customer's security question "who can log in, and with which rights?", and it is the base for later phases: in Phase 3 the app runs as `svc-app`, in Phase 6 groups control `sudo` and Docker access, and in Phase 7A Ansible automates joiners and leavers.

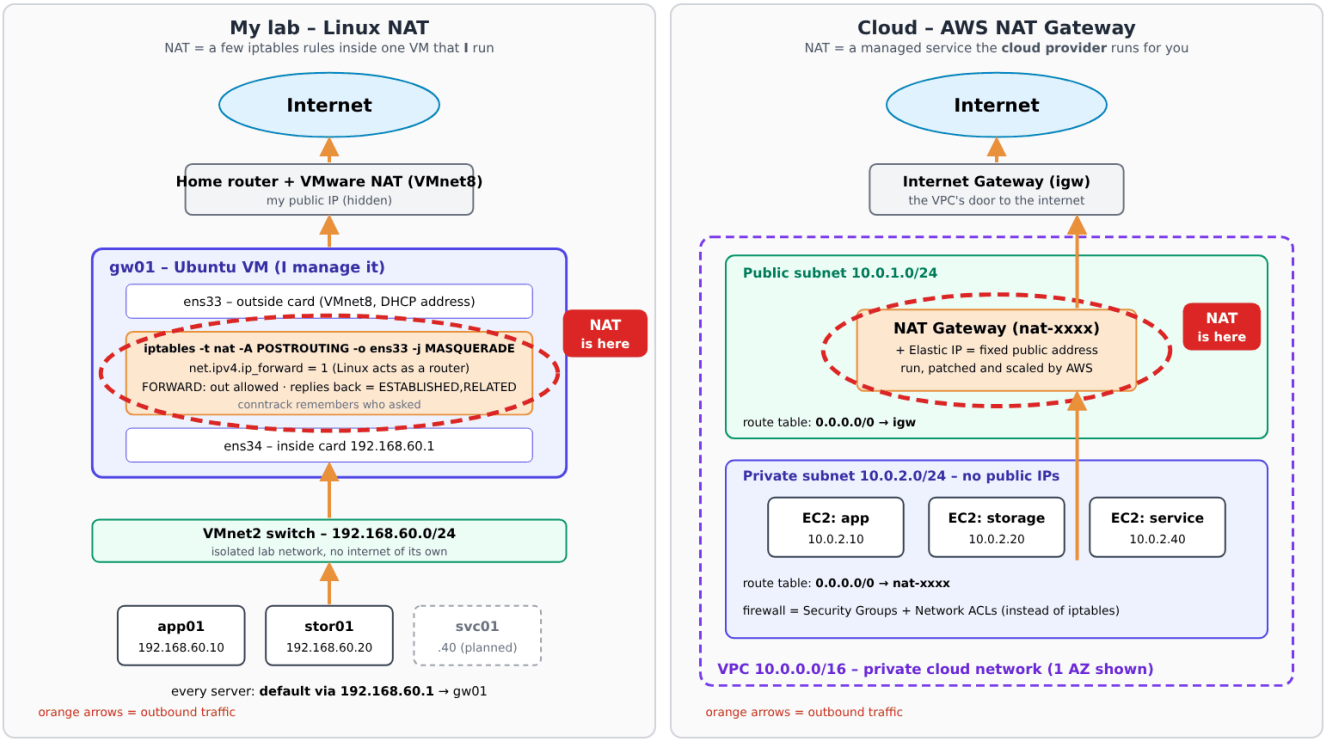
Step 1.8 complete.

13. Step 1.9 – Linux NAT vs Cloud NAT

💡 **In plain words:** NAT is the mailroom that swaps the private sender address for the company's public address on the way out and hands the replies back to the right person. In my lab I **build** that mailroom myself on gw01. In the cloud you **rent** it as a ready-made service.

Where does NAT live? Linux NAT (my lab) vs Cloud NAT (AWS)

Same job in both: private servers reach the internet, but the internet cannot start a connection to them. The red circle marks the NAT.



Same job: swap the private sender address for a public one on the way out, and let only the replies back in. **Difference:** in the lab I build, patch and keep the NAT alive; in the cloud the provider does, and charges per hour and per GB.

My lab piece	Cloud equivalent (AWS)	What it does
gw01 + iptables MASQUERADE	NAT Gateway	Rewrites the source address of outgoing traffic
ens33 / VMware NAT address	Elastic IP	The public address the internet sees
Home router / VMnet8	Internet Gateway	The door between the network and the internet
VMnet2 192.168.60.0/24	Private subnet	Servers with private addresses only
default via 192.168.60.1	Route table 0.0.0.0/0 → nat-xxxx	Sends unknown destinations to the NAT
iptables FORWARD rules	Security Groups + NACLs	Decides which traffic may pass

Muhammad Bilal Javed · Acme Analytics Linux Platform · Phase 1, Step 1.9

Where does NAT live? Linux NAT in my lab vs AWS NAT Gateway

Where the NAT sits

	My lab (gw01)	Cloud (AWS)
The NAT itself	3 kernel settings on one VM: `ip_forward = 1`, iptables `MASQUERADE`, `FORWARD` rules	A **NAT Gateway** placed in a **public subnet**
How servers find it	`default via 192.168.60.1` in netplan on each server	Private subnet route table `0.0.0.0/0 → nat-xxxx`
Public address	The address of gw01's outside card (VMware NAT / home router)	An **Elastic IP** attached to the NAT Gateway
Way out to the internet	Home router / VMnet8	**Internet Gateway** (public subnet route `0.0.0.0/0 → igw`)
Firewall	iptables `FORWARD` chain	Security Groups + Network ACLs

Comparison across providers

	gw01 (mine)	AWS NAT Gateway	Azure NAT Gateway	GCP Cloud NAT
Who runs it	Me	AWS (managed)	Azure (managed)	Google (managed)
Where	One VM	Public subnet; private route table `0.0.0.0/0 → nat-xxxx`	Attached to a subnet	Configured on a Cloud Router; no NAT VM at all (software-defined)
High availability	None – one VM is a single point of failure	Redundant inside one Availability Zone; one gateway per AZ for full resilience	Managed, zonal	Managed, distributed
Public IP	Address of the NAT adapter	Elastic IP	Public IP(s) or prefix	Auto-allocated or reserved IPs
Cost model	The VM I already run	Per hour + per GB processed	Per hour + per GB processed	Per hour + per GB processed

In three sentences

1. **What stays the same:** the job – outbound source translation, so private servers reach the internet while nobody on the internet can open a connection to them.
2. **What the cloud adds:** the provider runs, patches, scales and keeps the NAT available; in my lab, if gw01 is off, app01 and stor01 lose the internet.
3. **What I would watch:** cost – managed NAT is charged per hour **and per GB**, so heavy traffic (updates, backups, container image pulls) through it gets expensive; options such as VPC endpoints for cloud storage keep that traffic off the NAT.

Key understanding (in my own words)

> In my lab the NAT is not a box – it is three settings inside gw01: forwarding on, a MASQUERADE rule, and FORWARD rules that let replies back. Servers reach it because their default route points to gw01. In the cloud the same job is a managed NAT Gateway in a public subnet, reached through a route table entry and given a fixed Elastic IP; the provider keeps it running and bills per hour and per GB. Same job, different owner.

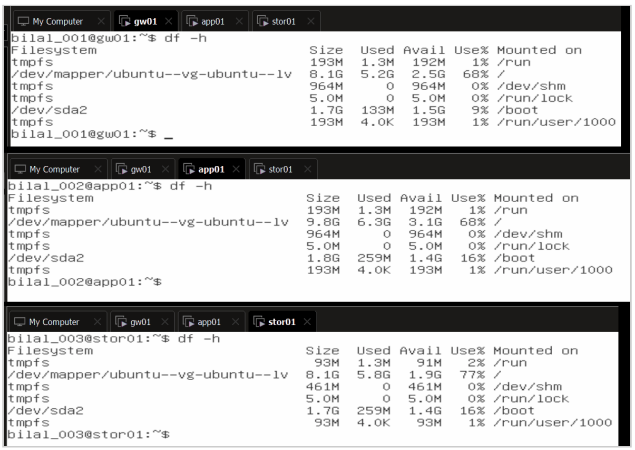
14. End of Phase 1 – updates and snapshot

All three VMs were updated (`sudo apt update && sudo apt -y upgrade`), shut down cleanly and given a VMware snapshot – the **restore point** for everything built in Phase 1.

VM	Snapshot name	Root disk	Used
gw01	`20-Network-gw01`	8.1 G	68 %
app01	`20-Network-app01`	9.8 G	68 %
stor01	`20-Network-stor01`	8.1 G	77 %

Description on every snapshot: *Phase 1 done: static IPs, NAT via gw01, hosts, SSH keys, users/groups, updates installed.*

📸 **Evidence – disk usage on gw01, app01, stor01 before the snapshot:**



df -h on all three VMs

💡 **Why:** a snapshot taken while the VM is **off** is clean and small (no memory saved). If Phase 2 (storage) breaks something, I return to this point in one click instead of rebuilding the network. stor01's root disk is the fullest, which is fine: backups will go to its separate data disks (`sdb`, `sdc`) with LVM in Phase 2.

Phase 1 complete ✅

15. Next steps

1. **Phase 0 leftover:** switch app01 and stor01 to server mode (`multi-user.target`).
2. **Phase 1A:** GLPI service desk – tickets, knowledge base and inventory on svc01.
3. **Phase 2:** storage – LVM on stor01's data disks, NFS share, backups with hard links, LVM snapshots.